

EXPENSE AND FINANCE TRACKER

1.ABSTRACT

The Expense Tracker is a web-based application developed to help users manage their financial activities efficiently. It enables users to record, track, and analyze their income and expenses in real time. The system is designed using modern web technologies such as React and TypeScript for the frontend, and Node.js with Express.js for the backend. Firebase Firestore is used as a NoSQL database for secure and real-time data storage, while Firebase Authentication provides secure login using Google and email.

The application offers features such as expense categorization, budget management, transaction history, and financial summaries. These features help users understand their spending patterns and make better financial decisions. The main objective of this project is to provide a simple, user-friendly, and efficient solution for personal financial management.

2. Introduction

In today's fast-paced digital world, managing personal finances efficiently has become increasingly important. Many individuals struggle to track their daily expenses, savings, and overall financial activities, which often leads to poor budgeting and financial instability. To address this problem, the Expense Tracker application is developed as a smart and user-friendly solution that helps users monitor and control their spending habits effectively.

The Expense Tracker is a modern web-based application built using advanced technologies such as TypeScript, React, HTML, and Tailwind CSS for the frontend, providing a responsive and interactive user interface. The backend is powered by Node.js and Express.js, ensuring smooth data handling and efficient server-side operations. Additionally, Firebase Firestore is used as a real-time database to store user data securely, while Firebase Authentication enables secure login through Google and email/password methods.

This application allows users to add, view, update, and delete their expenses, categorize spending, and analyze financial data through simple and intuitive interfaces. By offering real-time data synchronization and secure authentication, the system ensures both reliability and user privacy.

The main objective of this project is to simplify personal financial management by providing an easy-to-use platform that promotes better budgeting, financial awareness, and decision-making.

3.System Analysis

System analysis is a crucial phase in the development of the Expense Tracker application, where the requirements of the system are studied, analyzed, and clearly defined. This phase focuses on understanding user needs, identifying system functionalities, and determining how the system will operate efficiently using the chosen technologies.

1. Existing System

In the existing system, most individuals manage their expenses manually using notebooks, spreadsheets, or simple mobile notes. These methods have several limitations such as:

Lack of automation and real-time updates

High chances of human error in calculations

No proper categorization of expenses

Difficulty in tracking financial history

Limited data security and backup options

Because of these issues, users often fail to maintain accurate financial records and analyze their spending habits effectively.

2. Proposed System

The proposed system is a web-based Expense Tracker application designed to overcome the limitations of the existing system. It provides a digital platform where users can securely manage their financial activities.

Key features of the proposed system include:

User authentication using Google and email via Firebase

Real-time data storage using Firebase Firestore

Add, edit, delete, and categorize expenses

Responsive and interactive user interface using React and Tailwind CSS

Secure backend operations using Node.js and Express.js

Data synchronization across devices

This system improves accuracy, accessibility, and efficiency in managing expenses.

3. System Requirements

a) Functional Requirements:

User registration and login

Add new expense details (amount, category, date, description)

View list of expenses

Edit and delete existing expenses

Categorize expenses (food, travel, bills, etc.)

Display summary or reports of spending

b) Non-Functional Requirements:

User-friendly interface

Fast and responsive performance

Data security and privacy

Scalability for multiple users

Reliability and availability

4. Feasibility Study

a) Technical Feasibility:

The system is technically feasible as it uses widely adopted technologies like React, Node.js, and Firebase, which are reliable and easy to implement.

b) Economic Feasibility:

The project is cost-effective since Firebase provides free-tier services and the development tools used are open-source.

c) Operational Feasibility:

The system is easy to use and requires minimal training, making it suitable for all types of users.

5.SYSTEM ARCHITECTURE OVERVIEW

The system follows a client-server architecture:

Frontend: Built with React, TypeScript, HTML, and Tailwind CSS

Backend: Node.js with Express.

js handles API requests

Database: Firebase Firestore stores user and expense data

Authentication: Firebase Authentication manages user login

This architecture ensures smooth communication between the user interface and the database.

4.Requirement Specification

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: At least 500 MB free space
- Internet connection for accessing the application

Software Requirements

- **Frontend Technologies:** TypeScript, React, HTML, Tailwind CSS
- **Backend Technologies:** Node.js, Express.js
- **Database:** Firebase Firestore
- **Authentication:** Firebase Authentication (Google and Email)
- **Development Tools:** Visual Studio Code, Web Browser (Chrome/Edge)
- **Operating System:** Windows / macOS / Linux

User Requirements

- Basic knowledge of using web applications
- Internet access to use the system
- A Google account or email ID for authentication

5. System Design

System design defines how the Expense Tracker application is structured and how its components interact with each other. It provides a blueprint for implementing the system based on the requirements identified during the analysis phase.

1. Overview of System Design

The Expense Tracker follows a **client-server architecture** where the frontend, backend, and database work together to provide a seamless user experience.

- **Frontend (Client Side):** Built using TypeScript, React, HTML, and Tailwind CSS
- **Backend (Server Side):** Developed using Node.js and Express.js
- **Database:** Firebase Firestore (NoSQL database)
- **Authentication:** Firebase Authentication (Google and Email login)

2. Architecture Design

The system uses a **three-tier architecture**:

a) Presentation Layer (Frontend):

- Provides user interface for interaction
- Displays data such as expense lists and reports
- Sends user requests to backend APIs

b) Application Layer (Backend):

- Handles business logic and API requests
- Processes user input and communicates with Firestore
- Ensures secure data transactions

c) Data Layer (Database):

- Stores user details and expense records
- Provides real-time synchronization using Firebase Firestore

3. Module Design

The system is divided into the following modules:

- **Authentication Module**
 - Handles user login, registration, and logout
 - Supports Google and email/password authentication
- **Expense Management Module**
 - Add, update, delete, and view expenses
 - Manage expense categories
- **Dashboard Module**
 - Displays total expenses and summaries
 - Provides graphical or summarized reports
- **Database Module**
 - Stores and retrieves data from Firebase Firestore
 - Ensures data consistency and real-time updates

4. Data Flow Design (DFD - Level 0)

User → Frontend → Backend → Database → Backend → Frontend → User

Explanation:

- The user interacts with the React interface
- The frontend sends requests to the Express backend

- The backend processes the request and interacts with Firestore
- Data is returned back to the frontend and displayed to the user

5. Database Design (Firestore Structure)

Since Firebase Firestore is a NoSQL database, data is stored in collections and documents.

Collection: Users

- userId
- name
- email

Collection: Expenses

- expenseId
- userId
- amount
- category
- date
- description

6. User Interface Design

- Clean and responsive UI using Tailwind CSS
- Dashboard with summary of expenses
- Forms for adding and editing expenses
- Mobile-friendly layout with responsive design

7. Security Design

- Authentication handled by Firebase Auth
- Secure API endpoints in Express.js
- Data access restricted based on user identity
- HTTPS ensures safe data transmission

8. System Workflow

1. User logs in using Google or email
2. User adds or manages expenses
3. Data is sent to backend and stored in Firestore
4. Real-time updates reflect on the dashboard
5. User can view summaries and reports

6.SYSTEM DESCRIPTION

The Expense Tracker System is a web-based application designed to help users manage their financial activities efficiently. It enables users to track income and expenses, monitor spending patterns, and make better financial decisions.

Key Points:

- The system allows users to add, edit, delete, and view expense records in real time.
- Users can categorize expenses such as food, travel, bills, and others for better organization.
- It provides a dashboard that displays financial summaries and insights.
- The application supports budget management to control spending limits.
- It maintains a transaction history for tracking past expenses.
- The system uses Firebase Authentication to provide secure login using Google and email/password.
- Firebase Firestore is used as a NoSQL database for storing user and expense data securely.

- The frontend is developed using React, TypeScript, HTML, and Tailwind CSS for a responsive interface.
- The backend is implemented using Node.js and Express.js to handle API requests and business logic.
- Real-time data synchronization ensures that updates are instantly reflected across devices.
- The system ensures data security and privacy through authentication and secure data handling.

Overall, the Expense Tracker System provides a simple, efficient, and reliable solution for personal financial management.

7.SYSTEM ARCHITECTURE

The Expense Tracker System follows a client-server architecture, which is divided into three main layers: Presentation Layer, Application Layer, and Data Layer.

1. Presentation Layer (Frontend):

This layer is responsible for user interaction. It is developed using React, TypeScript, HTML, and Tailwind CSS. It provides a responsive and user-friendly interface where users can add, view, and manage their expenses.

2. Application Layer (Backend):

This layer handles the business logic and processes user requests. It is built using Node.js and Express.js. The backend receives requests from the frontend, processes them, and communicates with the database.

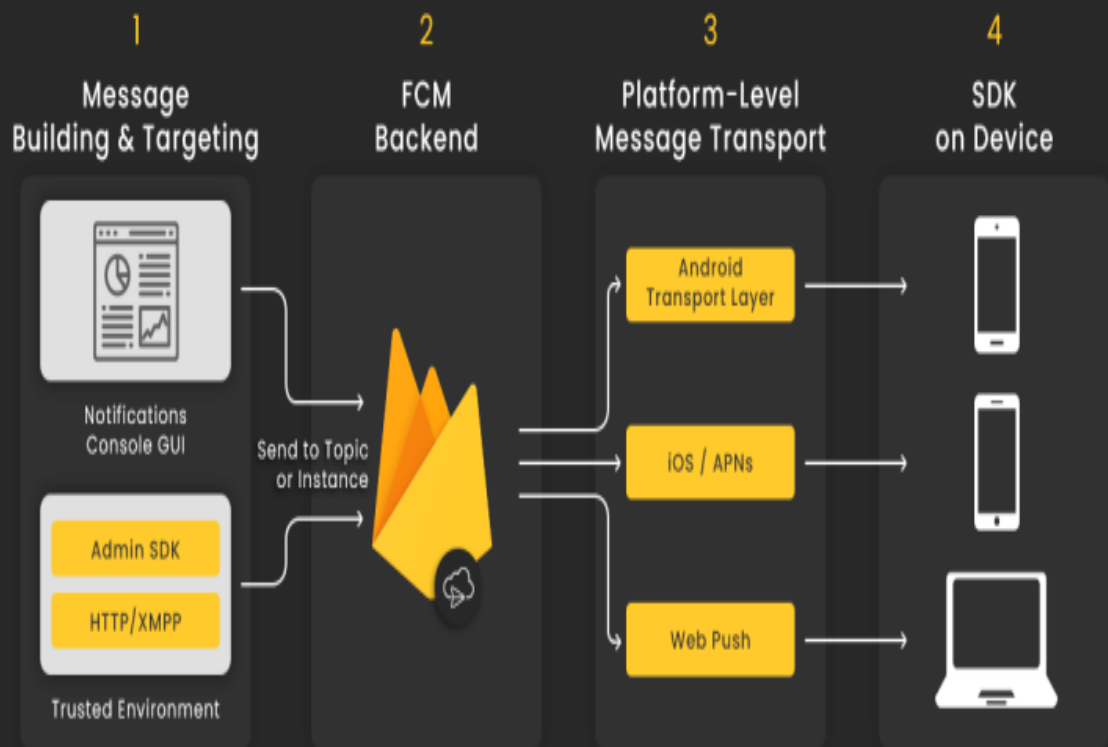
3. Data Layer (Database):

This layer stores and manages data. Firebase Firestore is used as a NoSQL database to store user and expense data. Firebase Authentication is used to securely manage user login and access control.

Working Process:

The user interacts with the frontend interface. The frontend sends requests to the backend APIs. The backend processes the data and stores or retrieves it **from Firebase Firestore**. **The results are then sent back to the frontend and displayed to the user.**

Firestore Architecture



8. TESTING AND IMPLEMENTATION

The Testing and Implementation phase ensures that the Expense Tracker System functions correctly, efficiently, and securely. Various testing methods are used to verify that all components of the system work as expected and meet user requirements. After successful testing, the system is deployed for real-time use, allowing users to manage their financial activities smoothly.

Key Points:

- Testing ensures the system is free from errors and bugs.
 - Unit testing checks individual components like forms and functions.
 - Integration testing verifies communication between frontend, backend, and database.
 - System testing ensures the complete application works correctly.
 - UI testing checks responsiveness and user-friendliness.
 - Security testing ensures safe user authentication and data protection.
-
- Implementation involves deploying the system for actual use.
 - Users can log in securely using Firebase Authentication.
 - Expenses can be added, updated, deleted, and viewed easily.
 - Data is stored securely in Firebase Firestore.
 - Real-time updates improve user experience.
 - The system is simple, efficient, and accessible through a web browser.

9.FORMS AND REPORTS

Forms and reports are important components of the Expense Tracker System. Forms are used to collect input data from users, while reports are used to display processed information in a meaningful way.

Forms:

The system provides different forms to allow users to enter and manage their financial data efficiently.

- **Login Form:**

Allows users to log in using email/password or Google authentication.

- **Add Expense Form:**

Used to enter details such as amount, category, date, and description of expenses.

- **Edit Expense Form:**

Allows users to modify existing expense records.

- Budget Form:

Enables users to set and manage monthly budget limits.

- Settings Form:

Used to update user preferences and account details.

These forms ensure easy data entry and improve user interaction with the system.

Reports:

Reports are used to present data in a structured and understandable format for analysis.

- Expense Summary Report:

Displays total expenses over a selected period.

- Category-wise Report:

Shows spending distribution across different categories like food, travel, and bills.

- Transaction History Report:

Lists all past transactions with details such as date, amount, and category.

- Budget Report:

Compares budget limits with actual expenses.

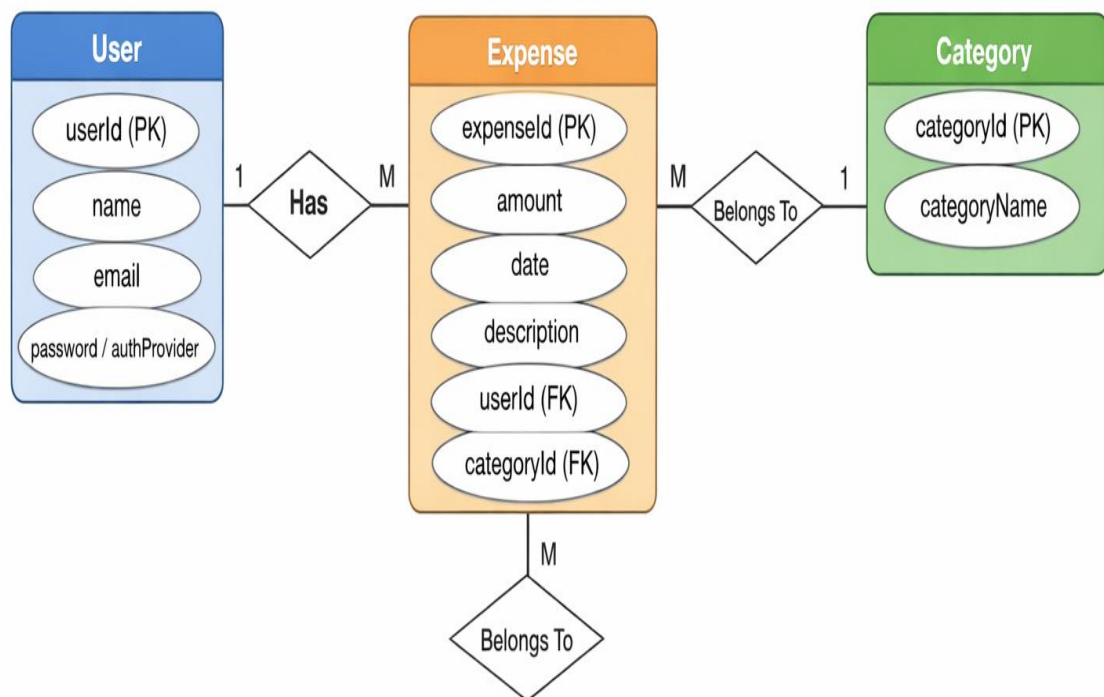
- Financial Dashboard:

Provides graphical representation of income, expenses, and savings.

These reports help users analyze their financial data and make better decisions.

10.ER DIAGRAM

Expense Tracker



API DESIGN

The backend of the Expense Tracker application provides RESTful API endpoints to manage expense data. These APIs allow the frontend to communicate with the server and perform operations such as adding, retrieving, and deleting expenses.

API Endpoints:

1. Add Expense

Method: POST

Endpoint: /api/expenses

Description: Adds a new expense to the database

Request Body:

- amount
- categoryId
- date
- description

2. Get All Expenses

Method: GET

Endpoint: /api/expenses

Description: Retrieves all expenses from the database

3. Delete Expense

Method: DELETE

Endpoint: /api/expenses/:id

Description: Deletes an expense based on its ID

These APIs ensure smooth communication between the frontend and backend, enabling real-time data management.

11.TABLE STRUCTURE

1.User table

Users Collection		
Field Name	Data Type	Description
userId	String	Unique user ID (Document ID)
name	String	Name of the user
email	String	User email address
authProvider	String	Google / Email login type
createdAt	Timestamp	Account creation time

2.category table

Categories Collection		
Field Name	Data Type	Description
categoryId	String	Unique category ID (Document ID)
userId	String	Reference to user
categoryName	String	Category name (Food, Travel, etc.)

3. Expense Table

Expenses Collection		
Field Name	Data Type	Description
expenseId	String	Unique expense ID (Document ID)
userId	String	Reference to user
categoryId	String	Reference to category
amount	Number	Expense amount
date	Timestamp	Expense date
description	String	Additional details
createdAt	Timestamp	Record creation time

4. Budget collection

Budgets Collection (Optional)		
Field Name	Data Type	Description
budgetId	String	Unique budget ID (Document ID)
userId	String	Reference to user
amount	Number	Budget amount
month	String	Month (e.g. Jan 2026)
createdAt	Timestamp	Record creation time

12. DESIGN LAYOUT

Login page

[← Back to Home](#)

Welcome Back

Access your wealth dashboard

EMAIL

 jasranjasran483@gmail.com

PASSWORD

By signing in, you agree to our [Terms](#) and [Privacy Policy](#).

Sign In

SOCIAL LOGIN

 Google

Your data is safe with us • Secure & Encrypted

New here? [Create account](#)

Transaction page

New Transaction

Expense

Income

TITLE

What was it for?

AMOUNT

₹ 0.00

DATE

30-03-2026

CATEGORY

Food

RECURRING

NONE

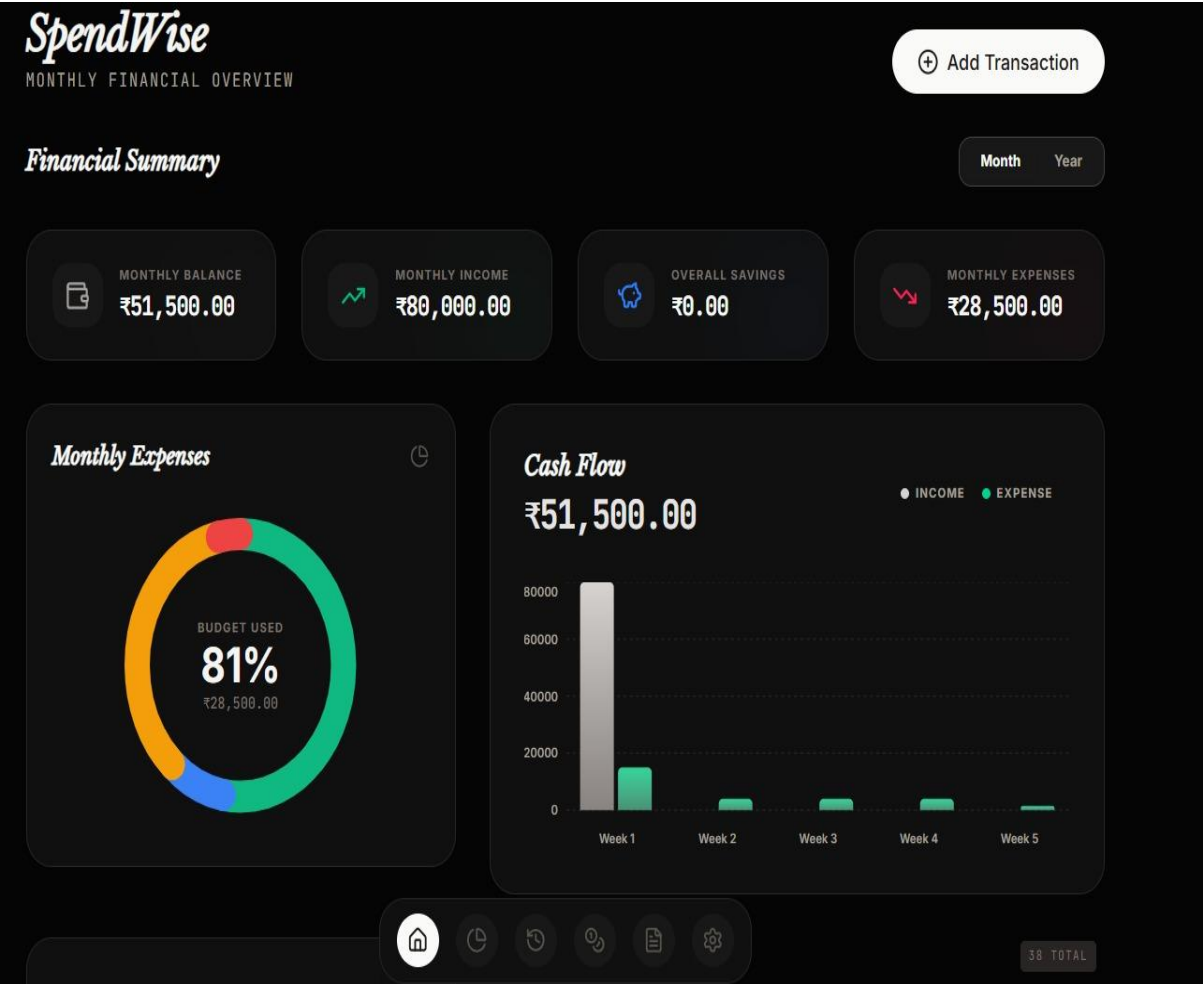
DAILY

WEEKLY

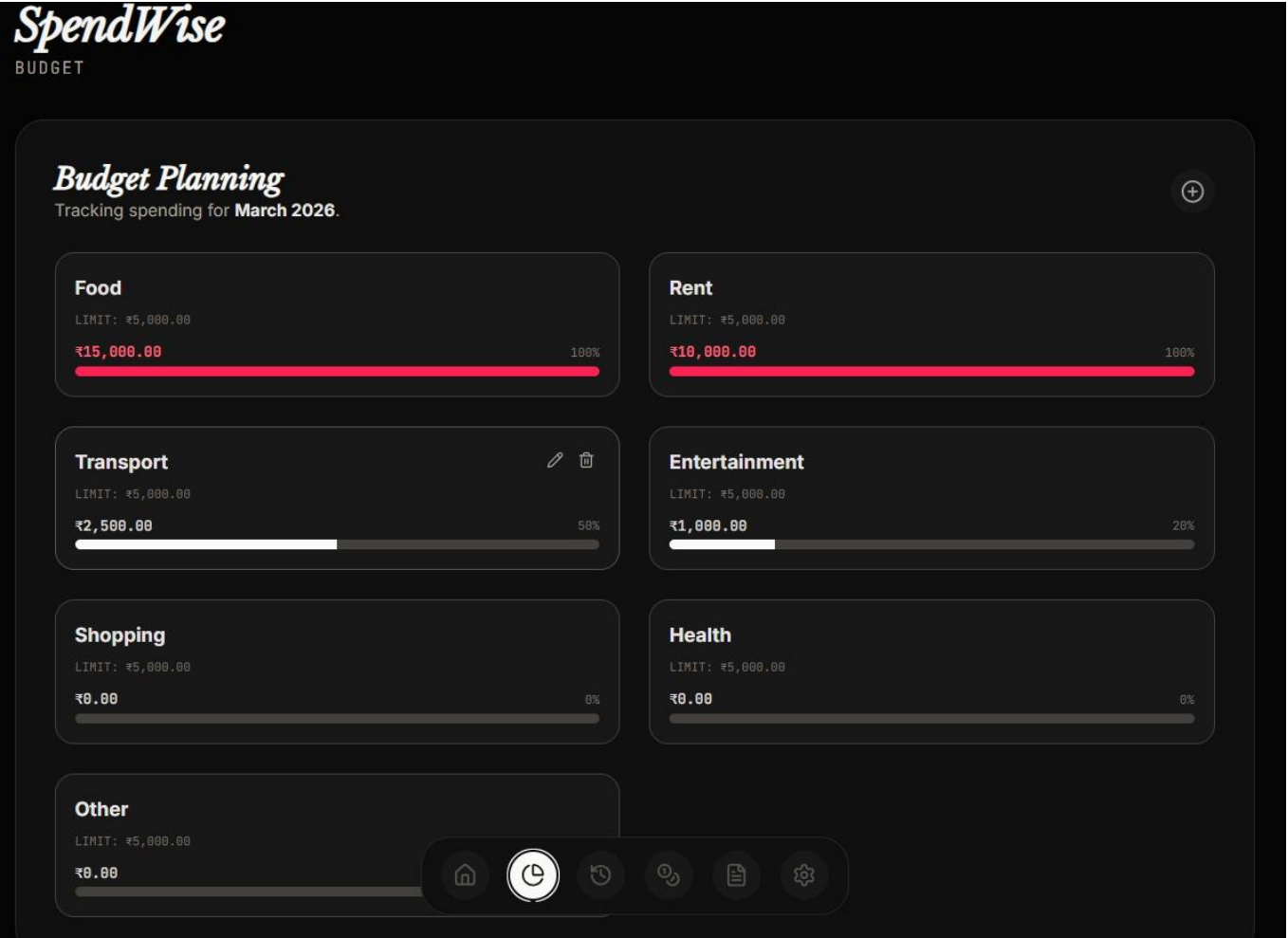
MONTHLY

Save Transaction

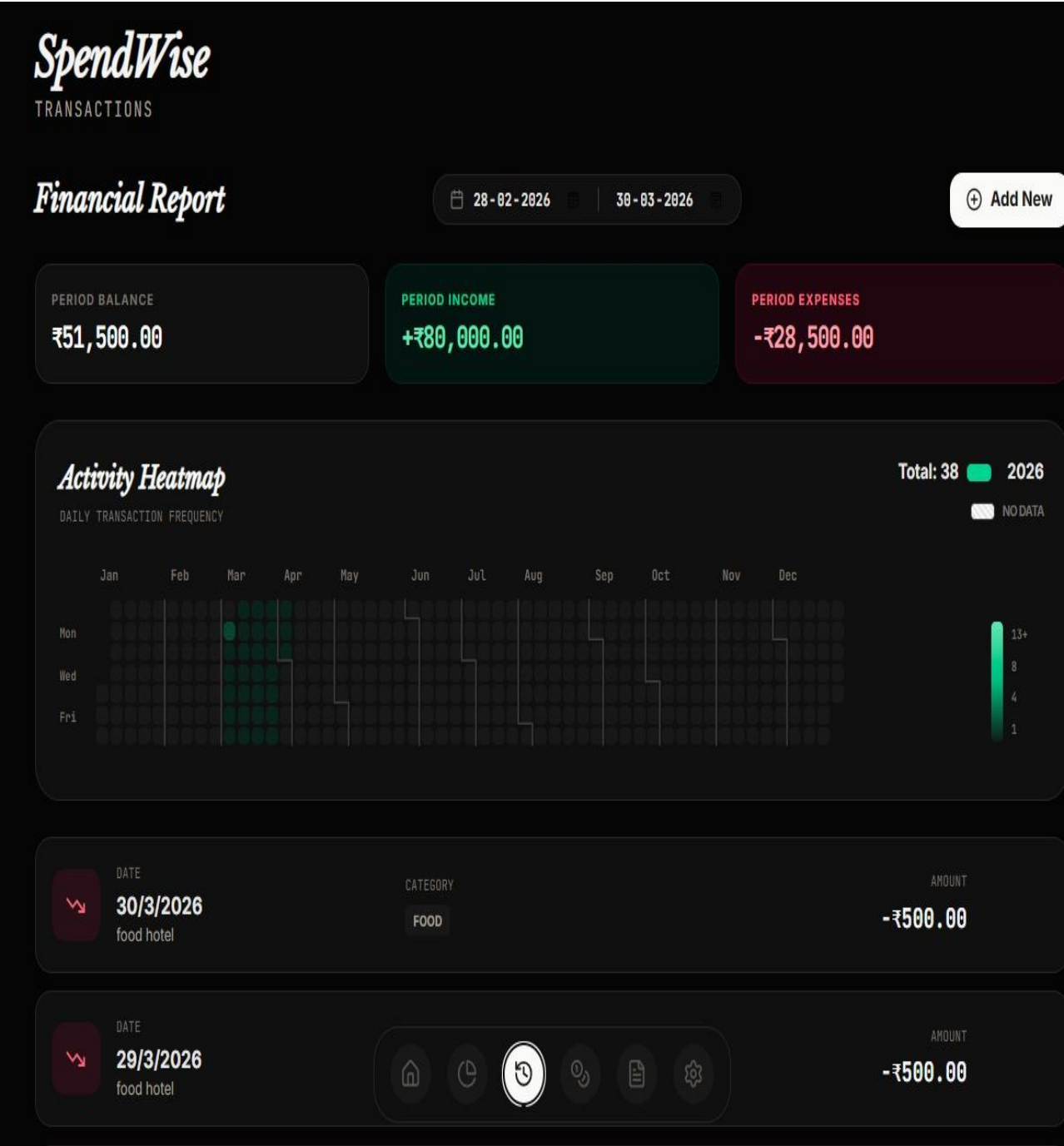
DASHBOARD



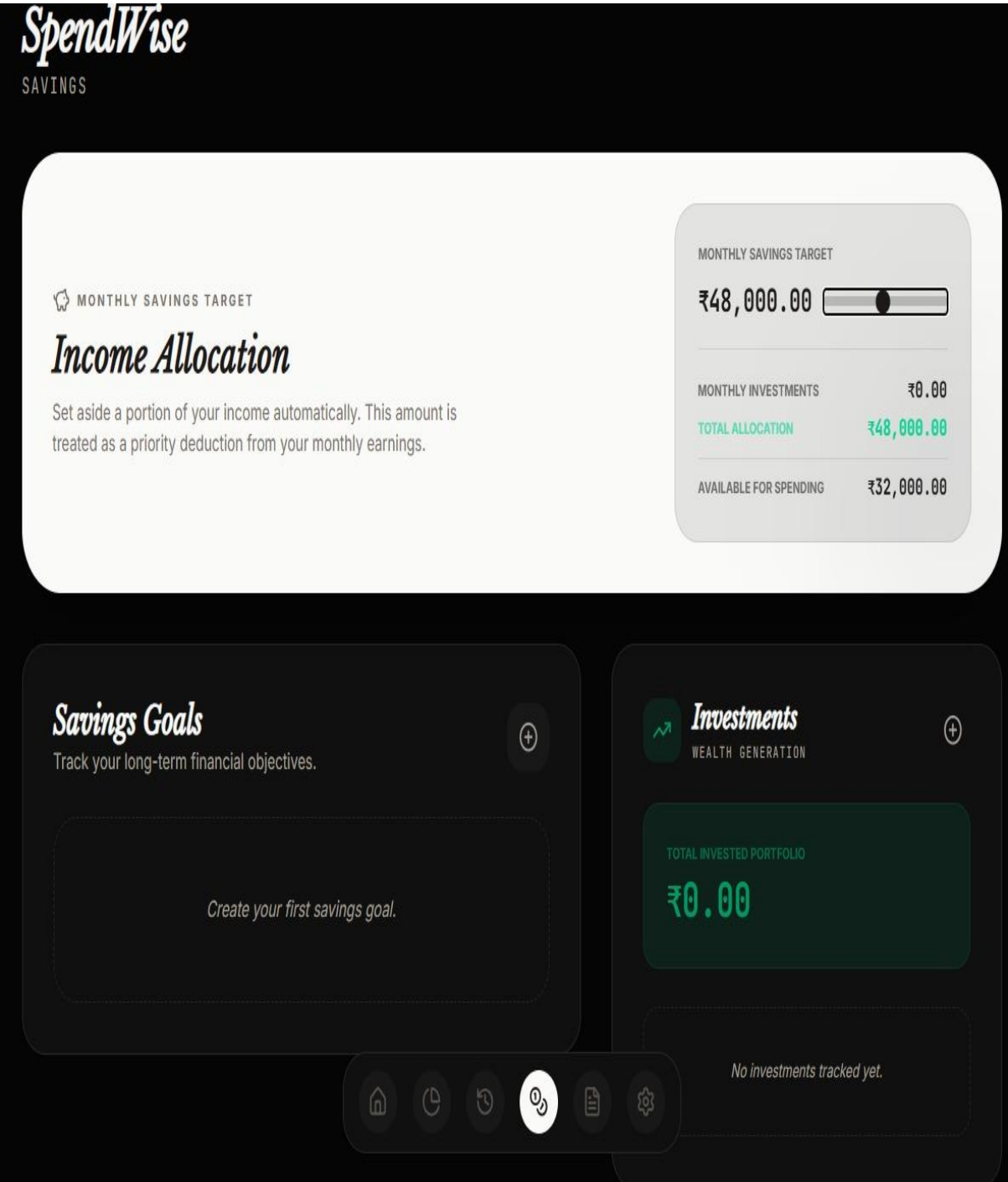
BUDGET



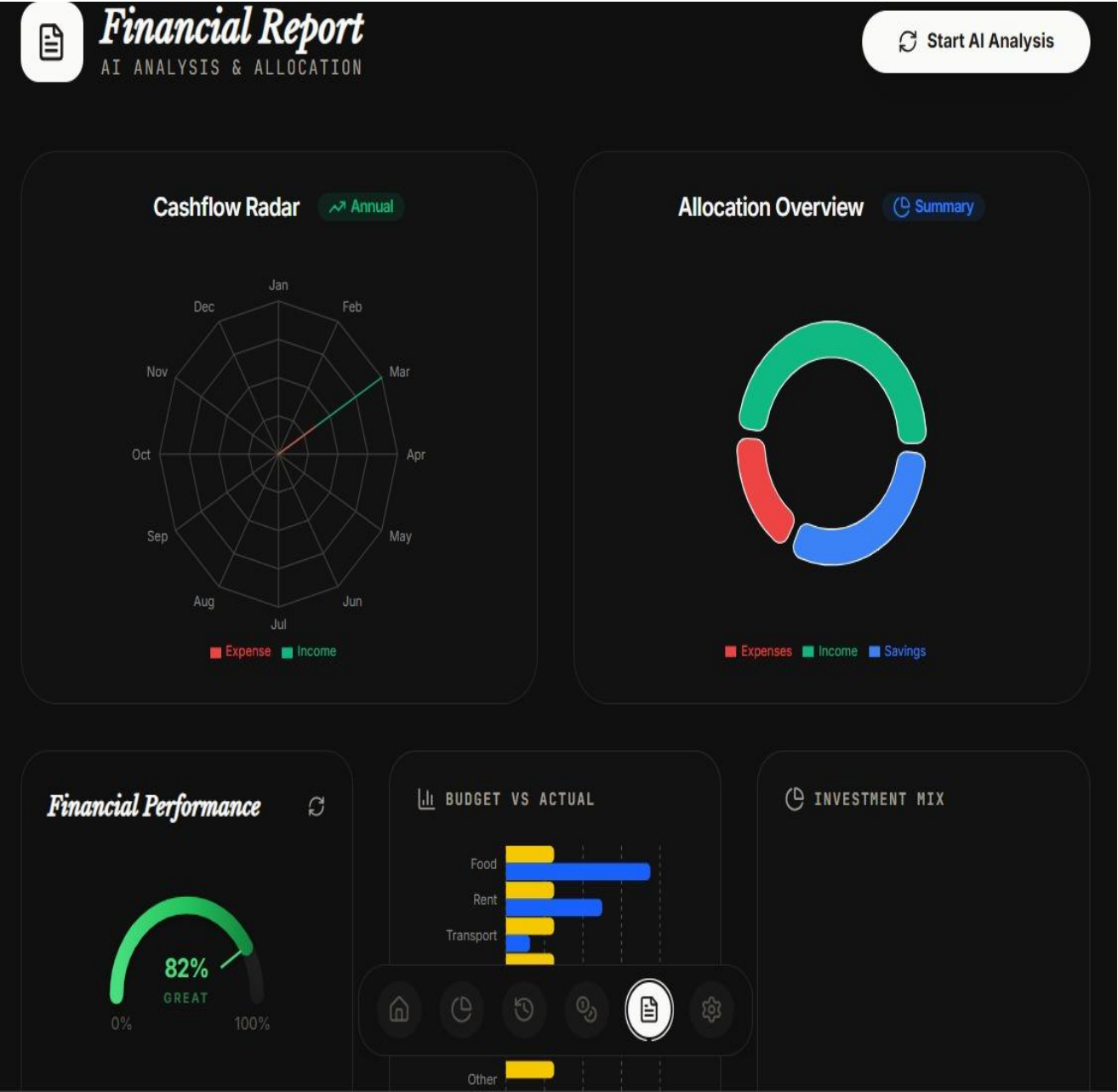
TRANSACTION HISTORY



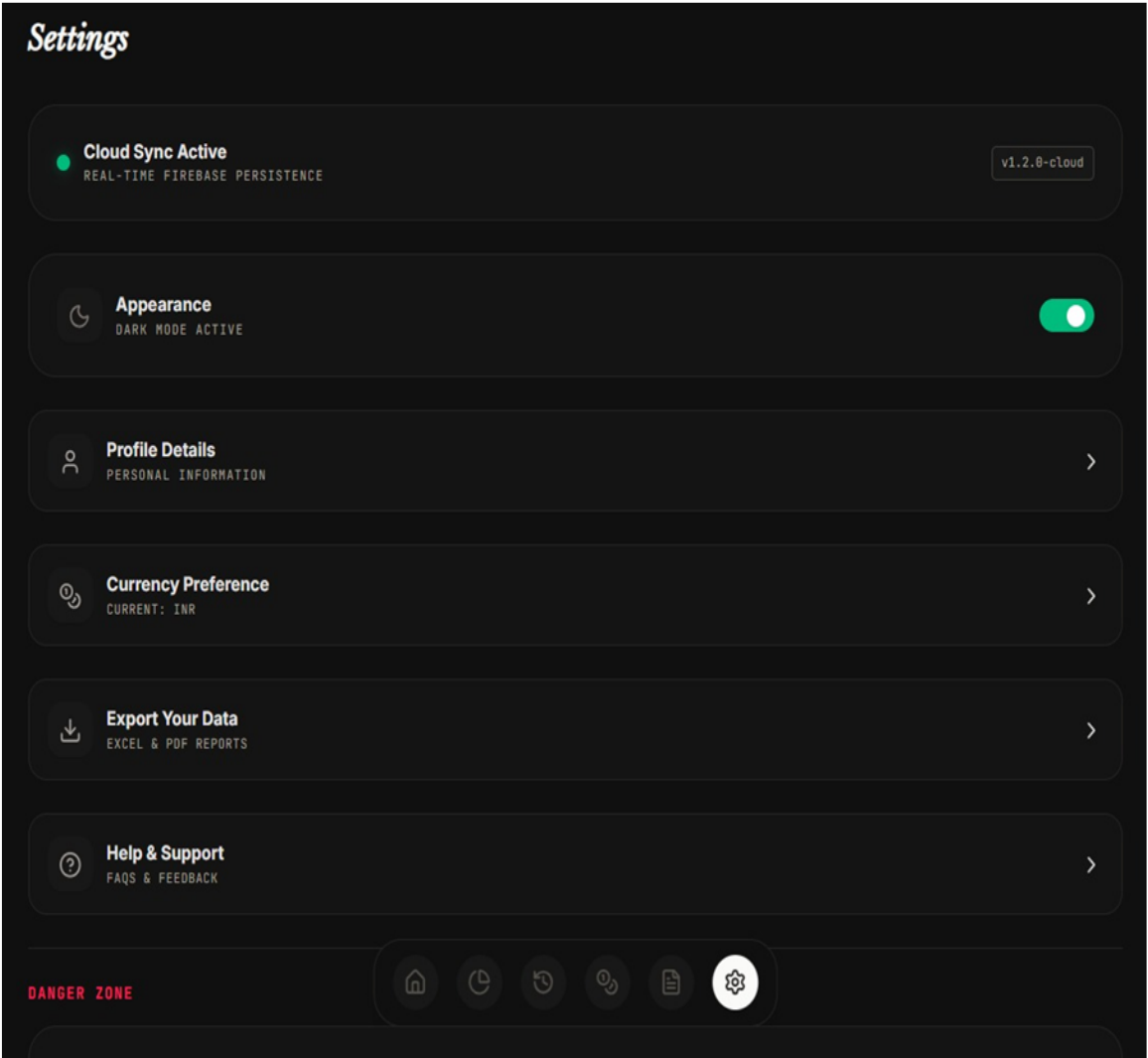
GOALS



FINANCIAL REPORT



SETTINGS



13.SOURCE CODE

FRONTEND ENTRY

```
import {StrictMode} from 'react';

import {createRoot} from 'react-dom/client';

import { BrowserRouter } from 'react-router-dom';

import App from './App.tsx';

import './index.css';

createRoot(document.getElementById('root')!).render(

  <StrictMode>

    <BrowserRouter>

      <App />

    </BrowserRouter>

  </StrictMode>,
```

INDEX.HTML

```
<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />


    <title>SpendWise | Smart Financial Tracking</title>

    <link rel="icon" type="image/x-icon" href="/favicon/favicon.ico" />

    <link rel="icon" type="image/png" sizes="32x32" href="public/favicon/favicon-32x32.png" />

    <link rel="icon" type="image/png" sizes="16x16" href="public/favicon/favicon-16x16.png" />

    <link rel="apple-touch-icon" sizes="180x180" href="public/favicon/apple-touch-icon.png" />


    <link rel="manifest" href="public/favicon/site.webmanifest" />


    <meta name="description" content="SpendWise - An AI-powered financial companion to track expenses, manage budgets, and visualize your wealth journey." />

    <meta name="keywords" content="finance tracker, expense manager, budget planning, AI financial advisor, SpendWise" />

    <meta name="author" content="Mohamed Fazil RM" />
```

```
<meta property="og:type" content="website" />
```

```
<meta property="og:title" content="SpendWise | AI-Powered Finance Manager" />
```

```
<meta property="og:description" content="Take control of your money with smart insights  
and automated budgeting." />
```

```
<meta property="og:image" content="/og-preview.png" />
```

```
<meta name="twitter:card" content="summary_large_image" />
```

```
<meta name="twitter:title" content="SpendWise" />
```

```
<meta name="twitter:description" content="Smart financial tracking for the modern era." />
```

```
<meta name="theme-color" content="#1c1917" />
```

```
</head>
```

```
<body>
```

```
<div id="root"></div>
```

```
<script type="module" src="/src/main.tsx"></script>
```

```
</body>
```

```
</html>
```

PACKAGE.JSON

```
{  
  "name": "react-example",  
  "private": true,  
  "version": "0.0.0",  
  "type": "module",  
  "scripts": {  
    "dev": "tsx server.ts",  
    "build": "vite build",  
    "preview": "vite preview",  
    "clean": "rm -rf dist",  
    "lint": "tsc --noEmit"  
  },  
  "dependencies": {  
    "@google/genai": "^1.29.0",  
    "@tailwindcss/vite": "^4.1.14",  
    "@vitejs/plugin-react": "^5.0.4",  
    "better-sqlite3": "^12.4.1",  
    "class-variance-authority": "^0.7.1",  
    "clsx": "^2.1.1",  
    "dotenv": "^17.2.3",  
    "express": "^4.21.2",  
    "firebase": "^12.10.0",
```

```
"framer-motion": "^12.35.2",  
"googleapis": "^171.4.0",  
"html2canvas": "^1.4.1",  
"jspdf": "^4.2.0",  
"jspdf-autotable": "^5.0.7",  
"lucide-react": "^0.546.0",  
"mathjs": "^15.1.1",  
"motion": "^12.23.24",  
"react": "^19.0.0",  
"react-dom": "^19.0.0",  
"react-markdown": "^10.1.0",  
"react-router-dom": "^7.13.1",  
"recharts": "^3.8.0",  
"tailwind-merge": "^3.5.0",  
"vite": "^6.2.0",  
"xlsx": "^0.18.5"  
},
```

```
"devDependencies": {  
  "@types/express": "^4.17.21",  
  "@types/node": "^22.14.0",  
  "autoprefixer": "^10.4.21",  
  "tailwindcss": "^4.1.14",  
  "tsx": "^4.21.0",  
  "typescript": "~5.8.2",
```

"vite": "^6.2.0"

}

}

BACKEND CODE :

SERVER.TS

```
import express from "express";

import cors from "cors";

import { initializeApp } from "firebase/app";

import { getFirestore, collection, addDoc, getDocs, deleteDoc, doc } from "firebase/firestore";


const app = express();

const PORT = 3000;


app.use(cors());

app.use(express.json());


// Firebase Configuration

const firebaseConfig = {

  apiKey: "YOUR_API_KEY",

  authDomain: "YOUR_PROJECT.firebaseio.com",

  projectId: "YOUR_PROJECT_ID",

  storageBucket: "YOUR_PROJECT.appspot.com",

  messagingSenderId: "XXXX",

  appId: "XXXX"

};
```

```
const firebaseApp = initializeApp(firebaseConfig);

const db = getFirestore(firebaseApp);


// API Routes


// Add Expense

app.post("/api/expenses", async (req, res) => {

  try {

    const data = req.body;

    const docRef = await addDoc(collection(db, "expenses"), data);

    res.json({ id: docRef.id });

  } catch (error) {

    res.status(500).json({ error: "Failed to add expense" });

  }

});


// Get Expenses

app.get("/api/expenses", async (req, res) => {

  try {

    const snapshot = await getDocs(collection(db, "expenses"));

    const expenses = snapshot.docs.map(doc => ({

      id: doc.id,

      ...doc.data()

    }));

  }

});
```

```
    res.json(expenses);

  } catch (error) {

    res.status(500).json({ error: "Failed to fetch expenses" });

  }

});
```

// Delete Expense

```
app.delete("/api/expenses/:id", async (req, res) => {

  try {

    const id = req.params.id;

    await deleteDoc(doc(db, "expenses", id));

    res.json({ success: true });

  } catch (error) {

    res.status(500).json({ error: "Failed to delete expense" });

  }

});
```

```
app.listen(PORT, () => {

  console.log(`Server running on http://localhost:\${PORT}`);

})
```

MAIN API FILE

```
import { Routes, Route } from "react-router-dom";

import HomePage from "../pages/HomePage";

import TransactionsPage from "../pages/TransactionsPage";

import BudgetPage from "../pages/BudgetPage";

import SettingsPage from "../pages/SettingsPage";


function App() {

  return (

    <div>

      <Routes>

        <Route path="/" element={<HomePage />} />

        <Route path="/transactions" element={<TransactionsPage />} />

        <Route path="/budget" element={<BudgetPage />} />

        <Route path="/settings" element={<SettingsPage />} />

      </Routes>

    </div>

  );

  setIsGoalFormOpen(true);

};


const handleSaveGoal = async () => {

  if (!newGoalName || !newGoalTarget) return;
```

```
const target = parseFloat(newGoalTarget);

const current = parseFloat(newGoalCurrent) || 0;

const goalData = {

  name: newGoalName,

  target,

  current,

  category: newGoalCategory || "General",

  userId: user?.uid

};

try {

  if (editingGoal) {

    await updateDoc(doc(db, "savings", editingGoal.id), goalData);

    setEditingGoal(null);

    showAlert("success", "Savings goal updated.");

  } else {

    await addDoc(collection(db, "savings"), goalData);

    showAlert("success", "New savings goal created!");

  }

} catch (err) {

  showAlert("error", "Failed to save goal.");

}

setIsGoalFormOpen(false);
```

```
setNewGoalName("");  
setNewGoalTarget("");  
setNewGoalCurrent("");  
setNewGoalCategory("");  
};
```

```
const handleDeleteGoal = async (id: string) => {  
  try {  
    await deleteDoc(doc(db, "savings", id));  
    showAlert("info", "Savings goal deleted.");  
  } catch (err) {  
    showAlert("error", "Failed to delete goal.");  
  }  
};
```

```
const handleClearData = async () => {  
  if (!user) return;  
  if (window.confirm("This will delete ALL your data permanently from the cloud. Are you  
sure?")) {  
    try {  
      const userId = user.uid;  
      const collections = ["transactions", "budgets", "savings", "recurring", "investments"];  
      for (const collName of collections) {  
        const q = query(collection(db, collName), where("userId", "==", userId));  
        const snapshot = await getDocs(q);
```

```

    const batch = writeBatch(db);

    snapshot.docs.forEach((doc) => batch.delete(doc.ref));

    await batch.commit();
  }

  await setDoc(doc(db, "users", userId), {

    currency: "INR",

    isDarkMode: false,

    monthlySavings: 0,

    userDetails: { name: user.displayName || "User", email: user.email || "" }

  });

  showAlert("success", "All your cloud data has been cleared.");

  window.location.reload();

} catch (err) {

  showAlert("error", "Failed to clear cloud data.");

}

}

};

```

```

const monthlyStats = useMemo(() => {

  const now = new Date();

  const firstDayOfMonth = new Date(now.getFullYear(), now.getMonth(),
1).toISOString().split('T')[0];

  const monthlyTransactions = transactions.filter(t => t.date >= firstDayOfMonth);

  const monthlyInvestments = investments

    .filter(i => i.date >= firstDayOfMonth)

```

```

    .reduce((acc, i) => acc + i.amount, 0);

    const income = monthlyTransactions.filter(t => t.type === "income").reduce((acc, t) => acc
+ t.amount, 0);

    const expenses = monthlyTransactions.filter(t => t.type === "expense").reduce((acc, t) =>
acc + t.amount, 0);

    const totalAllocation = monthlySavings + monthlyInvestments;

    const availableBalance = income - expenses - totalAllocation;

    return {

        income,

        expenses,

        savings: monthlySavings,

        investments: monthlyInvestments,

        totalAllocation,

        balance: availableBalance

    };

}, [transactions, monthlySavings, investments]);

const allStats = useMemo(() => {

    const income = transactions.filter(t => t.type === "income").reduce((acc, t) => acc +
t.amount, 0);

    const expenses = transactions.filter(t => t.type === "expense").reduce((acc, t) => acc +
t.amount, 0);

    return { income, expenses, balance: income - expenses };

```

```
}, [transactions]);
```

```
useEffect(() => {
```

```
  if (monthlyStats.totalAllocation > monthlyStats.income && monthlyStats.income > 0) {
```

```
    const timer = setTimeout(() => {
```

```
      showAlert("warning", `Allocation exceeds income (Allocation:
${formatCurrency(monthlyStats.totalAllocation)} | Income:
${formatCurrency(monthlyStats.income)}).`);
```

```
    }, 1000);
```

```
    return () => clearTimeout(timer);
```

```
  }
```

```
}, [monthlyStats.totalAllocation, monthlyStats.income, currency]);
```

```
const formatCurrency = (val: number) => {
```

```
  const curr = CURRENCIES.find(c => c.code === currency) || CURRENCIES[0];
```

```
  return new Intl.NumberFormat(curr.locale, {
```

```
    style: "currency",
```

```
    currency: curr.code,
```

```
    minimumFractionDigits: 2
```

```
  }).format(val);
```

```
};
```

```
const downloadBlob = (blob: Blob, filename: string) => {
```

```
  const url = URL.createObjectURL(blob);
```

```
  const link = document.createElement('a');
```

```
link.href = url;

link.download = filename;

document.body.appendChild(link);

link.click();

document.body.removeChild(link);

URL.revokeObjectURL(url);

};
```

```
const handleExportExcel = () => {

  try {

    const filtered = transactions.filter(t => t.date >= exportStartDate && t.date <=
exportEndDate);

    if (filtered.length === 0) {

      showAlert("warning", "No transactions found in the selected date range.");

      return;

    }

  }

}
```

```
const workbook = XLSX.utils.book_new();
```

```
// 1. Executive Summary Sheet
```

```
const income = filtered.filter(t => t.type === "income").reduce((acc, t) => acc + t.amount,
0);
```

```
const expenses = filtered.filter(t => t.type === "expense").reduce((acc, t) => acc +
t.amount, 0);
```

```

const summaryData = [
  ["SpendWise Financial Summary"],
  ["Period", `${exportStartDate} to ${exportEndDate}`],
  [],
  ["Metric", "Value", "Currency"],
  ["Total Income", income, currency],
  ["Total Expenses", expenses, currency],
  ["Net Balance", income - expenses, currency],
  [],
  ["Generated on", new Date().toLocaleString()]
];

const summarySheet = XLSX.utils.aoa_to_sheet(summaryData);
XLSX.utils.book_append_sheet(workbook, summarySheet, "Executive Summary");

// 2. Transactions Sheet

const txSheet = XLSX.utils.json_to_sheet(filtered.map(t => ({
  Date: t.date,
  Title: t.title,
  Type: t.type.toUpperCase(),
  Category: t.category,
  Amount: t.amount,
  Currency: currency
})));

XLSX.utils.book_append_sheet(workbook, txSheet, "Transactions");

```

```
// 3. Budgets Sheet
```

```
const budgetData = budgets.map(b => {  
  
  const spent = filtered  
  
    .filter(t => t.type === "expense" && t.category === b.name)  
  
    .reduce((acc, t) => acc + t.amount, 0);  
  
  return {  
  
    Category: b.name,  
  
    "Monthly Limit": b.limit,  
  
    "Total Spent in Period": spent,  
  
    "Remaining (vs Monthly)": b.limit - spent,  
  
    "Usage %": ((spent / b.limit) * 100).toFixed(2) + "%"   
  
  };  
  
});  
  
const budgetSheet = XLSX.utils.json_to_sheet(budgetData);  
  
XLSX.utils.book_append_sheet(workbook, budgetSheet, "Budgets");
```

```
// 4. Monthly Performance Sheet (Annual Breakdown)
```

```
const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct",  
"Nov", "Dec"];  
  
const currentYear = new Date().getFullYear();  
  
const monthlyPerformanceData = months.map((month, mIdx) => {  
  
  const row: any = { Month: month };  
  
  budgets.forEach(b => {  
  
    const spent = transactions
```

```

    .filter(t => {

        const d = new Date(t.date);

        return d.getFullYear() === currentYear && d.getMonth() === mIdx && t.category
        === b.name && t.type === "expense";

    })

    .reduce((acc, t) => acc + t.amount, 0);

    row[b.name] = spent;

});

return row;

});

const monthlyPerformanceSheet = XLSX.utils.json_to_sheet(monthlyPerformanceData);

XLSX.utils.book_append_sheet(workbook,    monthlyPerformanceSheet,    "Monthly
Performance");

```

// 5. Chart Data Sheet (Aggregated for easy charting)

```

const expenseByCategory = budgets.map(b => {

    const spent = filtered

    .filter(t => t.type === "expense" && t.category === b.name)

    .reduce((acc, t) => acc + t.amount, 0);

    return { Category: b.name, Amount: spent };

});

const chartDataSheet = XLSX.utils.json_to_sheet(expenseByCategory);

XLSX.utils.book_append_sheet(workbook, chartDataSheet, "Expense Chart Data");

```

// 6. Savings Sheet

```
const savingsData = savingsGoals.map(g => ({  
  Goal: g.name,  
  Category: g.category,  
  Target: g.target,  
  Current: g.current,  
  Remaining: g.target - g.current,  
  "Progress %": ((g.current / g.target) * 100).toFixed(2) + "%"  
}));
```

// 7. Investments Sheet

```
const investmentData = investments.map(inv => ({  
  Name: inv.name,  
  Type: inv.type,  
  Amount: inv.amount,  
  Date: inv.date,  
  Currency: currency  
}));  
  
const invSheet = XLSX.utils.json_to_sheet(investmentData);  
XLSX.utils.book_append_sheet(workbook, invSheet, "Investments");
```

```
const savingsSheet = XLSX.utils.json_to_sheet(savingsData);
```

```

XLSX.utils.book_append_sheet(workbook, savingsSheet, "Savings Goals");

const excelBuffer = XLSX.write(workbook, { bookType: 'xlsx', type: 'array' });

const blob = new Blob([excelBuffer], { type: 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' });

downloadBlob(blob,
`SpendWise_Financial_Report_${exportStartDate}_to_${exportEndDate}.xlsx`);

showAlert("success", "Excel report exported successfully.");

} catch (err) {

console.error("Excel Export Error:", err);

showAlert("error", "Failed to export Excel report.");

}

};

const handleExportPDF = async () => {

try {

const filtered = transactions.filter(t => t.date >= exportStartDate && t.date <= exportEndDate);

if (filtered.length === 0) {

showAlert("warning", "No transactions found in the selected date range.");

return;

}

setIsExporting(true);

```

```
// Small delay for UI feedback

await new Promise(resolve => setTimeout(resolve, 300));

const doc = new jsPDF();

const primaryColor: [number, number, number] = [28, 25, 23]; // stone-900

const secondaryColor: [number, number, number] = [120, 113, 108]; // stone-500


// PDF-safe currency formatter (uses code instead of symbol to avoid encoding issues)

const formatCurrencyPDF = (val: number) => {

  const curr = CURRENCIES.find(c => c.code === currency) || CURRENCIES[0];

  return new Intl.NumberFormat(curr.locale, {

    style: "currency",

    currency: curr.code,

    currencyDisplay: "code",

    minimumFractionDigits: 2

  }).format(val);

};


// Page 1: Cover & Summary

doc.setFont("helvetica", "bold");

doc.setFontSize(28);

doc.setTextColor(primaryColor[0], primaryColor[1], primaryColor[2]);

doc.text("SpendWise", 14, 30);
```

```
doc.setFont("helvetica", "italic");

doc.setFontSize(14);

doc.setTextColor(secondaryColor[0], secondaryColor[1], secondaryColor[2]);

doc.text("Financial Performance Report", 14, 40);


doc.setDrawColor(231, 229, 228); // stone-200

doc.line(14, 45, 196, 45);


doc.setFont("helvetica", "normal");

doc.setFontSize(10);

doc.text(`Reporting Period: ${exportStartDate} to ${exportEndDate}`, 14, 55);

doc.text(`Generated on: ${new Date().toLocaleString()}`, 14, 60);


// Summary Stats

const income = filtered.filter(t => t.type === "income").reduce((acc, t) => acc + t.amount,
0);

const expenses = filtered.filter(t => t.type === "expense").reduce((acc, t) => acc +
t.amount, 0);

const balance = income - expenses;


autoTable(doc, {

  startY: 70,

  head: [['Financial Metric', 'Amount']],

  body: [

    ['Total Income', formatCurrencyPDF(income)],
```

```

    ['Total Expenses', formatCurrencyPDF(expenses)],
    ['Net Cash Flow', formatCurrencyPDF(balance)],
  ],
  theme: 'striped',
  headStyles: { fillColor: primaryColor },
  styles: { font: 'helvetica', fontSize: 11 }
});

// Budgets Section

doc.addPage();

doc.setFontSize(16);

doc.setFont("helvetica", "bold");

doc.setTextColor(primaryColor[0], primaryColor[1], primaryColor[2]);

doc.text("Budget Analysis", 14, 20);

const budgetTableData = budgets.map(b => {

  const spent = filtered

    .filter(t => t.type === "expense" && t.category === b.name)

    .reduce((acc, t) => acc + t.amount, 0);

  return [

    b.name,

    formatCurrencyPDF(b.limit),

    formatCurrencyPDF(spent),

    ((spent / b.limit) * 100).toFixed(1) + "%"
  ]
});

```

```

];
});

autoTable(doc, {
  startY: 30,
  head: [['Category', 'Limit', 'Spent', 'Usage']],
  body: budgetTableData,
  headStyles: { fillColor: [68, 64, 60] as [number, number, number] }, // stone-700
});

// Savings Section

doc.addPage();

doc.setFontSize(16);

doc.setFont("helvetica", "bold");

doc.text("Savings Goals Progress", 14, 20);

const savingsTableData = savingsGoals.map(g => [
  g.name,
  g.category,
  formatCurrencyPDF(g.target),
  formatCurrencyPDF(g.current),
  ((g.current / g.target) * 100).toFixed(1) + "%"
]);

```

```
autoTable(doc, {  
  startY: 30,  
  head: [['Goal', 'Category', 'Target', 'Current', 'Progress']],  
  body: savingsTableData,  
  headStyles: { fillColor: [12, 10, 9] as [number, number, number] }, // stone-950  
});
```

```
// Investments Section
```

```
doc.addPage();  
doc.setFontSize(16);  
doc.setFont("helvetica", "bold");  
doc.setTextColor(primaryColor[0], primaryColor[1], primaryColor[2]);  
doc.text("Investment Portfolio", 14, 20);
```

```
const investmentTableData = investments.map(inv => [  
  inv.date,  
  inv.name,  
  inv.type,  
  formatCurrencyPDF(inv.amount)  
]);
```

```
autoTable(doc, {  
  startY: 30,  
  head: [['Date', 'Investment Name', 'Category', 'Amount']],
```

```
        body: investmentTableData,

        headStyles: { fillColor: [16, 185, 129] as [number, number, number] }, // emerald-500
color

    });
```

```
// Transactions Section
```

```
doc.addPage();

doc.setFontSize(16);

doc.setFont("helvetica", "bold");

doc.text("Detailed Transaction History", 14, 20);
```

```
const txTableData = filtered.map(t => [

    t.date,

    t.title,

    t.type.toUpperCase(),

    t.category,

    formatCurrencyPDF(t.amount)

]);
```

```
autoTable(doc, {

    startY: 30,

    head: [['Date', 'Title', 'Type', 'Category', 'Amount']],

    body: txTableData,
```

COMPONENT FILE

1.SUMMARYCARD.TSX

```
import React from "react";

import { cn } from "@src/types";

interface SummaryCardProps {

  title: string;

  amount: number;

  icon: React.ReactNode;

  accent: string;

  formatCurrency: (val: number) => string;

}

export function SummaryCard({ title, amount, icon, accent, formatCurrency }:
SummaryCardProps) {

  return (

    <div className="bg-white dark:bg-app-card p-6 rounded-3xl border border-stone-200
dark:border-stone-800 shadow-sm relative overflow-hidden group transition-colors duration-
300">

      <div className="flex items-center gap-4 relative z-10">

        <div className="p-3 bg-stone-50 dark:bg-app-card-alt rounded-2xl group-hover:scale-
110 transition-transform shrink-0">

          {icon}
```

```
</div>
```

```
<div className="min-w-0">
```

```
  <p className="text-[10px] uppercase tracking-widest font-bold text-stone-400
dark:text-stone-500 mb-0.5 truncate">{title}</p>
```

```
  <h2 className="text-xl font-mono font-bold tracking-tighter dark:text-stone-50
truncate">
```

```
    {formatCurrency(amount)}
```

```
</h2>
```

```
</div>
```

```
</div>
```

```
{/* Subtle background accent */}
```

```
<div className={cn(
```

```
  "absolute -right-4 -bottom-4 w-24 h-24 rounded-full opacity-[0.03] dark:opacity-[0.05]
blur-2xl",
```

```
  accent === "emerald" ? "bg-emerald-500" : accent === "rose" ? "bg-rose-500" : accent
=== "blue" ? "bg-blue-500" : "bg-stone-500"
```

```
  )} />
```

```
</div>
```

```
);
```

```
}
```

2.FINANCIALPERFORMANCE.TSX

```
import React from "react";

import { RefreshCw } from "lucide-react";

import { AnimatedRadialChart } from "@src/components/AnimatedRadialChart";

interface FinancialPerformanceProps {

  income: number;

  expenses: number;

  totalAllocation: number; // Savings + Investments

  summaryFilter: "Year" | "Month";

}

export const FinancialPerformance = ({

  income,

  expenses,

  totalAllocation,

  summaryFilter

}: FinancialPerformanceProps) => {

  // Scoring Logic

  const performanceScore = React.useMemo(() => {

    if (income === 0) return 0;

    const savingsRate = totalAllocation / income;
```

```

const savingsScore = Math.min((savingsRate / 0.2) * 50, 50);

const expenseRate = expenses / income;

const expenseScore = Math.max(0, (1 - expenseRate) * 50);

return Math.round(savingsScore + expenseScore);

}, [income, expenses, totalAllocation]);

```

```

return (

```

```

  <div className="bg-white dark:bg-app-card p-6 rounded-3xl border border-stone-200
dark:border-stone-800 shadow-sm flex flex-col items-center h-full">

```

```

    <div className="w-full flex items-center justify-between mb-4">

```

```

      <h3 className="font-serif italic text-lg font-bold dark:text-stone-50">

```

```

        Financial Performance

```

```

      </h3>

```

```

      <RefreshCw size={16} className="text-stone-400" />

```

```

    </div>

```

```

  <div className="flex-1 flex items-center justify-center py-2">

```

```

    <AnimatedRadialChart

```

```

      value={performanceScore}

```

```

      size={180}

```

```

      showLabels={true}

```

```

    />

```

```

  </div>

```

```

  <div className="mt-4 text-center space-y-2">

```

<p className="text-stone-400 dark:text-stone-500 text-[10px] font-medium leading-relaxed">

Score based on your {summaryFilter.toLowerCase()}ly savings and expense management.

</p>

</div>

</div>

);

};

3.TRANSACTIONHEATMAP.TSX

```
import React, { useMemo } from "react";

import { Transaction, cn } from "@src/types";

import { Tooltip, ResponsiveContainer } from "recharts";

interface TransactionHeatmapProps {

  transactions: Transaction[];

  formatCurrency: (val: number) => string;

}

export const TransactionHeatmap: React.FC<TransactionHeatmapProps> = ({ transactions,
formatCurrency }) => {

  const currentYear = new Date().getFullYear();

  const data = useMemo(() => {

    const dailyData: Record<string, { count: number; amount: number }> = {};

    transactions.forEach(t => {

      const dateStr = t.date; // YYYY-MM-DD

      if (!dailyData[dateStr]) {

        dailyData[dateStr] = { count: 0, amount: 0 };

      }

      dailyData[dateStr].count += 1;

    });

  });

}
```

```
    dailyData[dateStr].amount += t.amount;

  });

  return dailyData;
}, [transactions]);

const calendar = useMemo(() => {

  const days: { date: Date; dateStr: string; dayOfWeek: number; weekIndex: number; month:
number }[] = [];

  const startDate = new Date(currentYear, 0, 1);

  const endDate = new Date(currentYear, 11, 31);

  let currentDate = new Date(startDate);

  let weekIndex = 0;

  while (currentDate <= endDate) {

    const dayOfWeek = currentDate.getDay();

    const dateStr = currentDate.toISOString().split('T')[0];

    days.push({

      date: new Date(currentDate),

      dateStr,

      dayOfWeek,

      weekIndex,

      month: currentDate.getMonth()
```

```
});
```

```
if (dayOfWeek === 6) {
```

```
    weekIndex++;
```

```
}
```

```
currentDate.setDate(currentDate.getDate() + 1);
```

```
}
```

```
return days;
```

```
}, [currentYear]);
```

```
const maxAmount = useMemo(() => {
```

```
    const amounts = Object.values(data).map(d => (d as { amount: number }).amount);
```

```
    return Math.max(...amounts, 1);
```

```
}, [data]);
```

```
const getColor = (count: number) => {
```

```
    if (count === 0) return "bg-stone-100 dark:bg-app-card-alt";
```

```
    if (count <= 3) return "bg-emerald-100 dark:bg-emerald-900/30";
```

```
    if (count <= 7) return "bg-emerald-300 dark:bg-emerald-700/50";
```

```
    if (count <= 12) return "bg-emerald-500 dark:bg-emerald-500/70";
```

```
    return "bg-emerald-700 dark:bg-emerald-400";
```

```
};
```

```
const totalTransactions = useMemo(() => {  
  return Object.values(data).reduce((acc: number, curr) => acc + (curr as { count: number  
}).count, 0);  
}, [data]);
```

```
const monthLabels = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct",  
"Nov", "Dec"];
```

```
const dayLabels = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
```

```
// Calculate month label positions
```

```
const monthPositions = useMemo(() => {  
  const positions: { month: string; weekIndex: number }[] = [];  
  let lastMonth = -1;  
  calendar.forEach(day => {  
    if (day.month !== lastMonth) {  
      positions.push({ month: monthLabels[day.month], weekIndex: day.weekIndex });  
      lastMonth = day.month;  
    }  
  });  
  return positions;  
}, [calendar]);
```

```
// Calculate month boundary paths
```

```
const monthBoundaries = useMemo(() => {  
  const paths: string[] = [];
```

```
let lastMonth = -1;
```

```
calendar.forEach((day, i) => {
```

```
  if (day.month !== lastMonth && lastMonth !== -1) {
```

```
    // Month boundary found before this day
```

```
    const prevDay = calendar[i-1];
```

```
    const xBase = prevDay.weekIndex * 14;
```

```
    const yBase = prevDay.dayOfWeek * 14;
```

```
    // Jagged line path
```

```
    // Start from top of the grid at the week index of the first day of the new month
```

```
    const xStart = day.weekIndex * 14 - 2;
```

```
    let path = "";
```

```
    if (day.dayOfWeek === 0) {
```

```
      // Month starts on Sunday, simple vertical line
```

```
      path = `M ${xStart} 0 L ${xStart} 96`;
```

```
    } else {
```

```
      // Jagged line
```

```
      path = `M ${xStart} 0 L ${xStart} ${day.dayOfWeek * 14 - 2} L ${xStart + 14} ${day.dayOfWeek * 14 - 2} L ${xStart + 14} 96`;
```

```
    }
```

```
    paths.push(path);
```

```
  }
```

```
  lastMonth = day.month;
```

```

    });

    return paths;

  }, [calendar]);

  return (

    <div className="bg-white dark:bg-app-card p-6 rounded-3xl border border-stone-200
dark:border-stone-800 shadow-sm overflow-hidden transition-colors duration-300">

      <div className="flex items-start justify-between mb-6">

        <div>

          <h3 className="font-serif italic text-xl font-bold dark:text-stone-50">Activity
Heatmap</h3>

          <p className="text-[10px] font-mono uppercase text-stone-400 dark:text-stone-500
mt-1">Daily Transaction Frequency</p>

        </div>

        <div className="text-right space-y-2">

          <div className="flex items-center justify-end gap-4">

            <div className="flex items-center gap-2">

              <span className="text-sm font-bold text-stone-900 dark:text-stone-200">Total:
{totalTransactions.toLocaleString()}</span>

              <div className="w-6 h-3 rounded bg-emerald-900 dark:bg-emerald-400" />

            </div>

            <span className="text-sm font-bold text-stone-900 dark:text-stone-
200">{currentYear}</span>

          </div>

          <div className="flex items-center justify-end gap-2 text-[10px] font-bold text-stone-
400 dark:text-stone-500 uppercase tracking-tighter">

```

```
<div

  className="w-6 h-3 rounded border border-stone-200 dark:border-stone-700"

  style={{

    backgroundImage: 'repeating-linear-gradient(45deg, transparent, transparent 2px,
#e7e5e4 2px, #e7e5e4 4px)',

    backgroundColor: '#fafaf9'

  }}

/>

<span>No Data</span>

</div>

</div>

</div>
```

```
<div className="flex gap-6">

  <div className="flex-1 overflow-x-auto pb-2 scrollbar-hide">

    <div className="min-w-[780px] relative pt-6">

      { /* Month Labels */ }

      <div className="absolute top-0 left-10 right-0 h-4">

        {monthPositions.map((pos, i) => (

          <div

            key={i}

            className="text-[10px] font-mono font-bold text-stone-400 dark:text-stone-500
absolute"

            style={{ left: `${pos.weekIndex * 14}px` }}

          >
```

```

        {pos.month}

      </div>

    ))}

  </div>

<div className="flex gap-3">

  {/* Day Labels */}

  <div className="flex flex-col gap-[2px] pt-[2px]">

    {dayLabels.map((day, i) => (

      <div key={day} className="h-3 text-[9px] font-mono font-bold text-stone-400
dark:text-stone-500 flex items-center pr-1 w-6">

        {[1, 3, 5].includes(i) ? day : ""}

      </div>

    ))}

  </div>

  </div>

  {/* Grid Container */}

  <div className="relative flex-1 h-[100px]">

    {/* Cells */}

    {calendar.map((day) => {

      const dayData = data[day.dateStr] || { count: 0, amount: 0 };

      return (

        <div

          key={day.dateStr}

          className={cn(

```

```

        "absolute w-3 h-3 rounded-sm transition-all hover:ring-2 hover:ring-stone-300
dark:hover:ring-stone-600 cursor-help",

        getColor(dayData.count)

    )}

    style={{

        left: `${day.weekIndex * 14}px`,

        top: `${day.dayOfWeek * 14}px`

    }}

    title={` ${day.dateStr}:                ${formatCurrency(dayData.amount)}
(${dayData.count} trans.)`}

    />

    );

    )))

```

```

    { /* Month Boundary SVG */

    <svg className="absolute inset-0 w-full h-full pointer-events-none overflow-
visible">

    {monthBoundaries.map((path, i) => (

    <path

    key={i}

    d={path}

    fill="none"

    stroke="#d6d3d1"

    className="dark:stroke-stone-700"

    strokeWidth="1.5

```

```
        strokeLinecap="round"

        strokeLinejoin="round"

    />

    )})

</svg>

</div>

</div>

</div>

</div>
```

```
{/* Vertical Legend */}
```

```
<div className="flex items-center gap-2 pt-6 pr-4 shrink-0">

    <div className="relative h-[80px] w-3 rounded-sm overflow-hidden bg-gradient-to-t
from-emerald-50 via-emerald-400 to-emerald-900 dark:from-emerald-900/20 dark:via-
emerald-500 dark:to-emerald-300" />

    <div className="flex flex-col justify-between h-[80px] text-[9px] font-mono font-bold
text-stone-400 dark:text-stone-500 py-0.5">

        <span>13+</span>

        <span>8</span>

        <span>4</span>

        <span>1</span>

    </div>

</div>

</div>

</div>
```

FIREBASE.TS

```
import { initializeApp } from "firebase/app";

import { getFirestore } from "firebase/firestore";

import { getAuth, GoogleAuthProvider } from "firebase/auth";


const firebaseConfig = {

  apiKey: "YOUR_API_KEY",

  authDomain: "YOUR_PROJECT.firebaseio.com",

  projectId: "YOUR_PROJECT_ID",

  storageBucket: "YOUR_PROJECT.appspot.com",

  messagingSenderId: "XXXX",

  appId: "XXXX"

};


const app = initializeApp(firebaseConfig);


export const db = getFirestore(app);

export const auth = getAuth(app);

export const provider = new GoogleAuthProvider();
```

14.ADVANTAGES

- User-friendly and intuitive interface
- Real-time tracking of expenses and financial data
- Secure authentication using Firebase
- Accessible from anywhere with an internet connection
- Helps users improve budgeting and financial planning
- Reduces manual errors compared to traditional methods
- Provides clear insights into spending habits

15. CONCLUSION

The Expense Tracker System is a web-based application developed to help users manage their financial activities effectively. The system provides an easy and efficient way to record, track, and analyze income and expenses in real time.

The application is built using modern technologies such as React and TypeScript for the frontend, Node.js and Express.js for the backend, and Firebase for authentication and database management. These technologies ensure better performance, scalability, and security.

The system includes features such as transaction management, budget tracking, savings goals, and graphical analysis, which help users understand their spending patterns and make better financial decisions.

Overall, the project successfully demonstrates the implementation of a full-stack web application and highlights the importance of digital tools in personal financial management.

16.FUTURE ENHANCEMENT

- Development of a mobile application for Android and iOS platforms
- Integration of AI-based expense prediction and smart financial suggestions
- Support for multiple currencies for international users
- Implementation of notification alerts for budget limits and reminders
- Addition of data visualization features such as advanced charts and reports
- Offline mode support with automatic synchronization when internet is available

17.BIBLIOGRAPHY

1. React Official Documentation – <https://react.dev>
2. Node.js Documentation – <https://nodejs.org>
3. Express.js Guide – <https://expressjs.com>
4. Firebase Documentation – <https://firebase.google.com>
5. Google Sheets API Documentation – <https://developers.google.com/sheets/api>
6. Tailwind CSS Documentation – <https://tailwindcss.com>
7. JavaScript Documentation (MDN) – <https://developer.mozilla.org>